

Module 20 Notes:

Lesson : Advanced JavaScript

Map and Set:

- **Map:** Stores key-value pairs where keys can be of any type. It maintains insertion order and allows any data type as keys.
- **Set:** Stores unique values of any type, ensuring no duplicates. It does not have key-value pairs.

Spread and Rest:

- **Spread Operator (...):** Expands iterables (arrays, objects, strings) into individual elements. Useful for combining arrays/objects or passing arguments to functions.
- **Rest Operator (...):** Gathers multiple elements into an array, primarily used in function parameters or array destructuring.

Destructuring:

- **What is Destructuring?:** Allows extracting values from arrays or properties from objects into distinct variables using concise syntax {} for objects and [] for arrays.
- **Importance of Destructuring:**
 - Enhances code readability and reduces repetition.
 - Simplifies extracting values from complex data structures.
 - Supports default values and collecting remaining elements.

Default Parameter:

- Used in function parameters to assign default values if no argument or undefined is passed.

Array Destructuring:

- Extracts array elements into variables using []. Supports skipping elements, collecting remaining elements with rest (...), and assigning default values.

Object Destructuring:

- Extracts object properties into variables with matching names using {} syntax. Allows default values and renaming variables.

Call, Apply, and Bind Methods:

- **Call Method:** Invokes a function with a specified this context and individual arguments.
- **Apply Method:** Similar to call, but accepts arguments as an array or array-like object.
- **Bind Method:** Creates a new function with a specified this context, useful for function reusability.

Hoisting in JavaScript:

- **Definition:** Hoisting moves function and variable declarations to the top of their scope during compilation.
- **Behavior with var:** Variables declared with var are hoisted and initialized with undefined.
- **Behavior with let and const:** They are hoisted but remain uninitialized until their declaration is reached, resulting in a "Temporal Dead Zone" if accessed earlier.

Temporal Dead Zone:

- **Concept:** The period between entering scope and variable declaration where accessing let and const variables throws a ReferenceError.
- **Prevention:** Encourages best practices by enforcing variable declaration before use, reducing runtime errors.

Usage Examples:

- **Call Method:** Useful for borrowing methods between objects or functions.
- **Apply Method:** Suitable for passing arrays as function arguments.
- **Bind Method:** Creates functions with predetermined arguments and this context, aiding in function composition.

Practical Application:

- Understanding these methods and hoisting principles is crucial for writing robust JavaScript code.
- Helps avoid common pitfalls like variable scoping issues and enhances code clarity and maintainability.